

④ ServeHTTP メソッドの宣言

ServeHTTP メソッドの処理は、HTTP ヘッダの設定と Execute メソッドの呼び出しです。HTTP ヘッダには、送信するデータが HTML で文字コードが UTF-8 であることを設定しています。

Execute メソッドは、テンプレートと構造体を組み合わせた HTML をクライアントに送信します。Execute メソッドでエラーが生じた場合は、panic 関数でパニックを発生させます。

また、ServeHTTP メソッド内でパニックが発生した場合は、メソッドを呼び出した net/http パッケージ側でパニックを回復させます。そして回復した後、クライアントの接続を中断しスタックトレースを出力します。

プログラムの実行中に「http://localhost:4000/hello」や「http://localhost:4000/goodbye」を開くと、10 章と同じ結果が表示されます。

第 13 章 ◀

簡単な Web アプリケーション の作成②：○×ゲームの作成

複合型に関する文法要素を一通り紹介したので、ここで実際に配列や構造体などを使用して Web アプリケーションを作成してみましょう。本章で作成する Web アプリケーションは○×ゲームです。

13.1 入力画面の作成

本章で作成する○×ゲームは、入力画面にHTMLフォームのボタンを9個用意し、プレイヤーがこれらのボタンを押すことでゲームを進める形にします。「○」のプレイヤーと「×」のプレイヤーの手番は、ボタンを押すたびに入れ替わることにします。

まず初めに、入力画面の表示のみを行うWebアプリケーションを作成します。この入力画面には、12章で紹介したHTMLテンプレートエンジンを使用します。

▶ 13.1.1 入力画面のテンプレートファイル

9個のボタンを表示する入力画面のテンプレートファイルは、次のとおりです。

```
<html><head>
<title>マルバツゲーム</title>
<style type="text/css">
  table {border:3px solid; text-align:center; vertical-align:middle}
  td {border:3px solid; width:50px; height:50px}
  button {width:30px; height:30px}
</style>
</head><body>
  {/{* $.Turnは手番 */}}
  <div>[{($.Turn)}の手番]</div><br>
  <form action="/game" method="post">
    <table>
      {/{* $.Boardは盤面のデータ ([3][3]string型) */}}
      {(range $row, $rows := $.Board)}
      <tr>
        {(range $col, $cell := $rows)}
        <td><button></button></td>
      {(end)}
    </tr>
  {(end)}
</table>
</form>
<a href="/game">最初に戻る</a>
</body></html>
```

このテンプレートファイルを、プログラムを実行するディレクトリに「game.tmpl」という名前

で保存してください。

このテンプレートファイルは、「\$.Turn」と「\$.Board」を使用しています。「\$.Turn」はプレイヤーの手番であり「○」または「×」を設定します。一方、「\$.Board」は、図13.1のように盤面の状態を表す3×3の2次元配列([3][3]string)です。

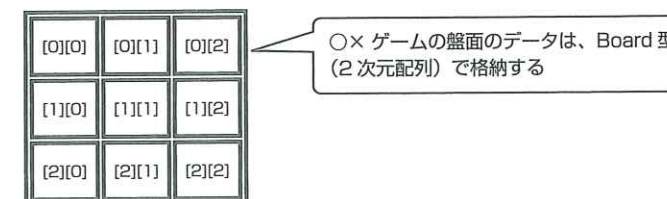


図 13.1 Board (2次元配列)

テンプレート側で配列をループするには「{{range 変数名 1, 変数名 2 := 配列}} ... {{end}}」と記述します。この記法の動作は、Go言語にあるfor文のrangeと基本的に同じです。このループによってテンプレートは、9個(3×3個)のボタンを持つHTMLを作成します。

▶ 13.1.2 入力画面のソースコード

Webアプリケーションのプログラムは、このテンプレートファイル「game.tmpl」にTurnフィールドとBoardフィールドを持つ構造体を組み合わせてクライアントに送信します。入力画面を表示するコードは次のとおりです。

```
package main

import (
  "html/template"
  "net/http"
)

// 盤面などで使用する文字列
const maru, batsu = "○", "×"

// Board型の宣言 ... ①
type Board [3][3]string

// ViewData型の宣言 ... ②
type ViewData struct {
  Turn string // 手番
```

```

    Board *Board // 盤面
}

// テンプレートの設定 ... ③
var tmpl *template.Template = template.Must(template.ParseFiles("game.tmpl"))

// Executeメソッドの宣言 ... ④
func (v *ViewData) Execute(w http.ResponseWriter) {
    // HTMLをクライアント（ブラウザ）に送信する
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    if err := tmpl.Execute(w, v); err != nil {
        panic(err)
    }
}

// gameHandle関数の宣言 ... ⑤
func gameHandle(w http.ResponseWriter, r *http.Request) {
    turn := maru    // 手番
    board := &Board{} // 盤面
    v := ViewData{turn, board}
    v.Execute(w)
}

func main() {
    http.HandleFunc("/game", gameHandle)

    if err := http.ListenAndServe(":4000", nil); err != nil {
        panic(err)
    }
}

```

まずは入力画面を表示するため、テンプレートと組み合わせる構造体の設定のみを行っています。フォームの入力値については、まだ処理していません。以下に各処理の詳細を説明します。

①Board 型の宣言

盤面の状態を表す Board 型を宣言します。Board 型は文字列を要素とする 3×3 の 2 次元配列型です。前ページの図 13.1 のように 1 次元目の配列を行（横）、2 次元目の配列を列（縦）として扱います。

②ViewData 型の宣言

テンプレートに設定する構造体型を ViewData 型として宣言します。Turn フィールドはプレイヤーの手番、Board フィールドは盤面の状態を表します。

③テンプレートの設定

変数 tmpl にテンプレートを設定しています。読み込むファイル名は「game.tmpl」です。

④Execute メソッドの宣言

ViewData 型に Execute メソッドを作成しています。Execute メソッド内の処理は、HTTP ヘッダの設定と tmpl.Execute メソッドの呼び出しです。HTTP ヘッダには、送信するデータが HTML であり、文字コードが UTF-8 であることを設定しています。

tmpl.Execute メソッドは、HTML をクライアント（ブラウザ）に送信します。なお、このとき送信する HTML は、テンプレートと ViewData 型の値を組み合わせたものです。

⑤gameHandle 関数の宣言

「http://localhost:4000/game」を開いたときに動作する関数です。ViewData 型の値を作成し、その値を元にした HTML を Execute メソッドで送信しています。

このプログラムを、「game.tmpl」を保存したディレクトリで実行してください。プログラムの実行中に「http://localhost:4000/game」を開くと、図 13.2 のような入力画面が表示されます。この段階では、フォームの入力値を処理していないため、ボタンを押しても画面は変化しません。



図 13.2 入力画面

13.2 入力値の処理

前項で入力画面を作成したので、次に入力値の処理を作成します。この○×ゲームは、「○」のプレイヤーと「×」のプレイヤーが交互に入力します。そのため、ボタンを押すたびにプレイヤーの手番を交代し、また、どのプレイヤーがボタンを押したかを盤面に記録する必要があります。

ここでは、手番の変更と盤面の記録のために HTML フォームを使用します。手番と盤面のデータは、毎回表示する HTML の中に埋め込みます。画面のボタンを押すと、図 13.3 のように埋め込んだデータを HTML フォームでサーバに送信します。

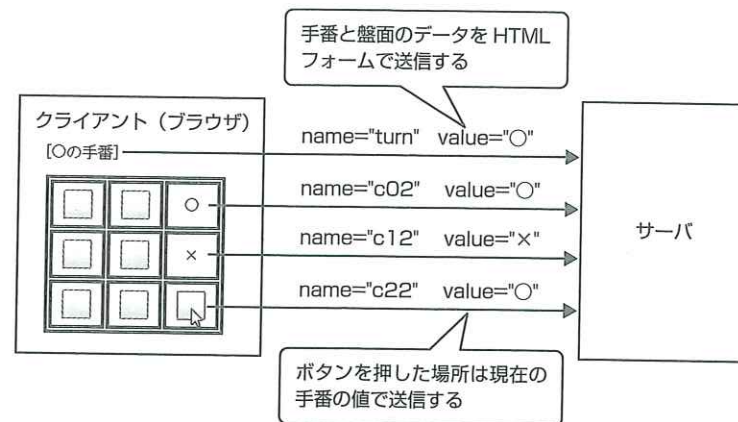


図 13.3 手番と盤面のデータ (HTML フォーム)

▶ 13.2.1 入力値の処理のテンプレートファイル

手番と盤面のデータを送信するため、「13.1.1」で作成した「game.tmpl」ファイルの form タグを次のように変更します。

```
<form action="/game" method="post">
  {/{* 手番は、inputタグのhiddenで送信 */}}
  <input type="hidden" name="turn" value="{{$.Turn}}">
  <table>
    {{(range $row, $rows := $.Board)}}
    <tr>
      {{(range $col, $cell := $rows)}}
      <td>
        {{if $cell}} {/{* "○" または "×" の場合 */}}
        {{ $cell }}
```

```
<input type="hidden" name="c{{ $row }}{{ $col }}" value="{{ $cell }}">
{{else}} {/{* " " の場合 */}}
<button name="c{{ $row }}{{ $col }}" value="{{ $.Turn }}"></button>
{{end}}
</td>
{{end}}
</tr>
{{end}}
</table>
</form>
```

手番のデータは input タグの name 属性「turn」、盤面のデータは input タグと button タグの name 属性「c00」～「c22」で送信します。c の後に続く数値は、盤面の行番号と列番号を表します。また、いくつかの input タグでは、type 属性に「hidden」を指定しています。HTML フォームは、この「hidden」を指定するとブラウザの画面上に何も表示せずにデータを送信できます。

盤面のデータが「○」または「×」の場合は、その値を画面に表示して、「hidden」を指定した input タグでサーバに送信します。また、盤面のデータが「」（初期値）の場合は、button タグでボタンを表示します。このとき button タグの value 属性には、現在の手番の値を指定します。

▶ 13.2.2 入力値の処理のソースコード

入力値を処理するコードは次のとおりです。「13.1.2」のコードから import 宣言と gameHandle 関数を変更しています。また、新規に boardFormValue 関数、nextTurnMap 変数、および turnFormValue 関数を追加しています。

```
import (
  "html/template"
  "net/http"
  "strconv" // strconvパッケージのインポートを追加 ... ①
)

// gameHandle関数の宣言 ... ②
func gameHandle(w http.ResponseWriter, r *http.Request) {
  _, nextTurn := turnFormValue(r) // 手番の入力値を取得する
  board := boardFormValue(r)      // 盤面の入力値を取得する

  // 値を設定してHTMLを送信する
```

```

v := ViewData{nextTurn, board}
v.Execute(w)
}

// boardFormValue関数の宣言 (盤面の値を取得) ... ③
func boardFormValue(r *http.Request) *Board {
    var board Board
    for row, rows := range board {
        for col, _ := range rows {
            // 盤面のname属性「c00」～「c22」を作成
            name := "c" + strconv.Itoa(row) + strconv.Itoa(col)
            // 盤面の各項目を取得
            board[row][col] = r.FormValue(name)
        }
    }
    return &board
}

// 変数nextTurnMapに、次の手番を取得するマップを設定 ... ④
var nextTurnMap = map[string]string{
    maru:batsu,
    batsu: maru,
    "":maru, // 「」の場合、ゲーム開始時として「○」を取得
}

// turnFormValue関数の宣言 (手番の値を取得) ... ⑤
func turnFormValue(r *http.Request) (string, string) {
    turn := r.FormValue("turn") // 現在の手番を取得
    nextTurn := nextTurnMap[turn] // マップを使用して次の手番を取得
    return turn, nextTurn
}

```

ここでは、追加した HTML フォームの入力値を取得して、それを ViewData 構造体に設定しています。各処理の詳細は次のとおりです。

① strconv パッケージのインポートを追加

import 宣言に strconv パッケージを追加しています。これは boardFormValue 関数で strconv.Itoa 関数を使用するためです。strconv.Itoa 関数は、文字列を数値に変換する関数です。

② gameHandle 関数の宣言

ViewData 構造体に設定する値を変更しています。設定する値は、turnFormValue 関数、boardFormValue 関数がフォームの値をもとに算出します。

③ boardFormValue 関数の宣言 (盤面の値を取得)

フォームから送信された「c00」～「c22」の値を取得して、変数 board の各要素に設定しています。変数 board に設定される値は「○」「×」「」のいずれかであり、盤面の各要素の状態を表します。戻り値は変数 board のアドレスです。

④ 変数 nextTurnMap に、次の手番を取得するマップを設定

変数 nextTurnMap に、現在の手番から次の手番を取得するマップを設定しています。このマップは turnFormValue 関数で使います。

⑤ turnFormValue 関数の宣言 (手番の値を取得)

フォームから送信された「turn」の値を取得します。この値は現在の手番を表し、「○」「×」「」のいずれかです。1 番目の戻り値は現在の手番、2 番目の戻り値は次の手番になります。取得した値が「○」または「×」の場合は、次の手番 (「○」であれば「×」、「×」であれば「○」) を返します。また、取得した値が「」の場合はゲーム開始時と見なして、先手である「○」を返します。

プログラムの実行中に「http://localhost:4000/game」を開いて、ボタンを押してみてください。

図 13.4 のように「×」の手番に交代して入力した値を表示します。

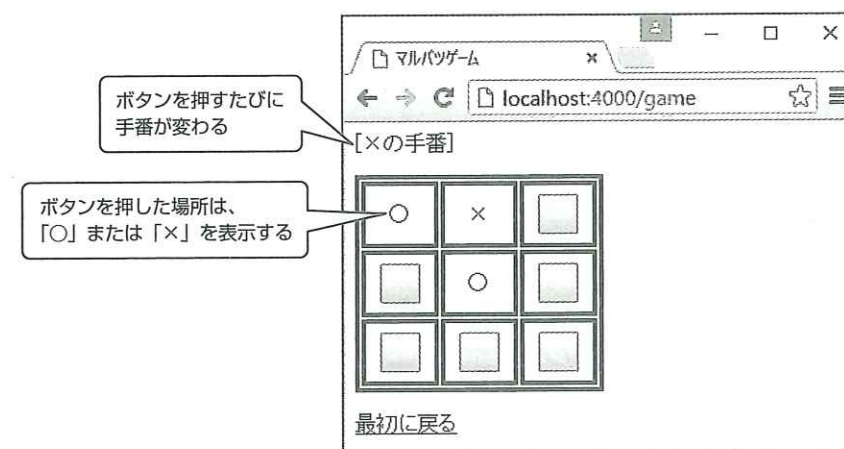


図 13.4 入力値の表示

13.3 勝敗と引き分けの判定

ここまでの処理で、プレイヤーの手番の交代と盤面の記録ができるようになりました。最後に勝敗と引き分けの判定を作成します。○×ゲームで勝敗が付くのは、一直線上に同じ種類の記号(○または×)が3つそろった場合です。引き分けになるのは、すべての場所の入力が終わり、これ以上ゲームを進められない場合です。

▶ 13.3.1 判定処理のテンプレートファイル

勝敗と引き分けを表示するために、「13.2.1」の「game.tmpl」ファイルを変更します。変更する箇所は、プレイヤーの手番を表示していた div タグです。div タグの変更は次のとおりです。

```
<div>
  {{if $.Win}}
    [{{$.Winner}}の勝ち]
  {{else if $.Draw}}
    [引き分け]
  {{else}}
    [{{$.Turn}}の手番]
  {{end}}
</div>
```

このテンプレートでは新規に「\$.Win」「\$.Winner」「\$.Draw」を使用しています。「\$.Win」には勝敗が付いた場合に true、付かない場合に false を設定します。また、「\$.Winner」には勝者の記号を「○」または「×」で設定します。「\$.Draw」には引き分けの場合に true、引き分け以外の場合に false を設定します。「\$.Win」と「\$.Draw」がいずれも false であれば、これまでと同じようにプレイヤーの手番を表示します。

▶ 13.3.2 判定処理のソースコード

テンプレートの変更に合わせたコードは次のとおりです。「13.2.2」のコードから ViewData 型と gameHandle 関数を変更しています。また、変数 winBoardIndexArray と Board 型の win メソッド、draw メソッド、および setBar メソッドを追加しています。

```
// ViewData型の宣言 … ①
type ViewData struct {
```

```
Turn string // 手番
Board *Board // 盤面
Win bool // 勝敗が付いた場合にtrueを設定
Draw bool // 引き分けの場合にtrueを設定
Winner string // 勝者を設定
}

// gameHandle関数の宣言 … ②
func gameHandle(w http.ResponseWriter, r *http.Request) {
    turn, nextTurn := turnFormValue(r) // 手番の入力値を取得する
    board := boardFormValue(r) // 盤面の入力値を取得する

    // 勝敗、引き分け、勝者の変数宣言
    win, draw, winner := false, false, ""

    // turnが「-」の場合、ゲーム開始時とする
    if turn != "-" { // ゲーム開始時以外
        win = board.win(turn) // 勝敗の判定

        if win { // 勝敗が付いた場合
            winner = turn // 勝者を設定
            board.setBar() // 未入力の項目に「-」を設定
        } else { // 勝敗が付かない場合
            draw = board.draw() // 引き分けの判定
        }
    }

    // 値を設定してHTMLを送信する
    v := ViewData(nextTurn, board, win, draw, winner)
    v.Execute(w)
}
```

```
// 変数winBoardIndexArrayに、勝敗判定用の構造体の2次元配列を設定 … ③
var winBoardIndexArray = [...]struct{ row, col int }{
    // 横(行)の判定
    {{0, 0}}, {{0, 1}}, {{0, 2}},
    {{1, 0}}, {{1, 1}}, {{1, 2}},
    {{2, 0}}, {{2, 1}}, {{2, 2}},
    // 縦(列)の判定
    {{0, 0}}, {{1, 0}}, {{2, 0}},
```

```

    {{0, 1}, {1, 1}, {2, 1}},
    {{0, 2}, {1, 2}, {2, 2}},
    // 斜めの判定
    {{0, 0}, {1, 1}, {2, 2}},
    {{0, 2}, {1, 1}, {2, 0}},
}

// winメソッドの宣言 (勝敗の判定) ... ④
func (b *Board) win(turn string) bool {
    for _, w := range winBoardIndexArray {
        // 3個すべてがそろった場合、勝利と判定
        if (b[w[0].row][w[0].col] == turn) &&
            (b[w[1].row][w[1].col] == turn) &&
            (b[w[2].row][w[2].col] == turn) {
            return true
        }
    }
    return false
}

```

```

// drawメソッドの宣言 (引き分けの判定) ... ⑤
func (b *Board) draw() bool {
    for row, rows := range b {
        for col, _ := range rows {
            if b[row][col] == "" {
                return false // 未入力の項目がある場合、ゲームを続行
            }
        }
    }
    return true // 未入力の項目がない場合、引き分け
}

```

```

// setBarメソッドの宣言 ("-" の設定) ... ⑥
func (b *Board) setBar() {
    for row, rows := range b {
        for col, _ := range rows {
            if b[row][col] == "" {
                b[row][col] = "-" // 未入力の項目は "-" を設定
            }
        }
    }
}

```

```

}
}

```

上記のコードで、勝敗と引き分けの判定結果を ViewData 構造体に設定しています。各処理の詳細は次のとおりです。

① ViewData 型の宣言

ViewData 型に Win フィールド、Draw フィールド、および Winner フィールドを追加しています。Win フィールドには勝敗が付いた場合に true、Draw フィールドには引き分けの場合に true を設定します。また、Winner フィールドには勝者を設定します。

② gameHandle 関数の宣言

ViewData 構造体のフィールドに値を設定しています。ゲーム開始時 (最初に画面を表示したとき) は勝敗や引き分けの判定をしません。ゲーム開始時以外の場合に勝敗の判定を行います。

勝敗の判定で勝者が決まった場合は勝者を設定し、未入力の項目に "-" を設定します。一方、勝者が決まらない場合は、引き分けの判定を行います。

③ 変数 winBoardIndexArray に、勝敗判定用の構造体の 2 次元配列を設定

変数 winBoardIndexArray に、勝敗判定用の値を構造体の 2 次元配列で格納します。O×ゲームで「勝ち」となるのは、直線上に「O」または「×」が 3 つそろったときだけです。

配列の要素である「struct { row, col int }」型は Board 型のインデックスを表し、そのインデックスを 3 つ持つ「[3]struct{ row, col int }」型は直線上にそろう必要がある 3 箇所を表します。「[...][3]struct{ row, col int }」型の変数 winBoardIndexArray は、勝敗を判定する直線上の項目をすべて持つ形になります。

たとえば、配列 1 つあたりの値「{{0, 0}, {0, 1}, {0, 2}}」は、Board 型のインデックスを 3 つ持ちます。このとき、この 3 つのインデックスが同じ直線上の項目を表しています。

④ win メソッドの宣言 (勝敗の判定)

Board 型の win メソッドは、引数の手番が勝利したときに true を返します。具体的には、変数 winBoardIndexArray をループして直線上の 3 つの項目が手番と一致したときに、勝利したと判定します。

⑤ draw メソッドの宣言 (引き分けの判定)

Board 型の draw メソッドは、引き分けとなった場合に true を返します。引き分けとなる条件は未入力の項目がないことです。それ以外は引き分けとせず、ゲームを続行します。

⑥ setBar メソッドの宣言 ("-" の設定)

Board 型の setBar メソッドは、勝敗が付いた場合に未入力の項目に "-" を設定します。

「http://localhost:4000/game」を開いて、勝敗が付くケースや引き分けのケースを試してみてください。図 13.5 や図 13.6 のように表示されます。

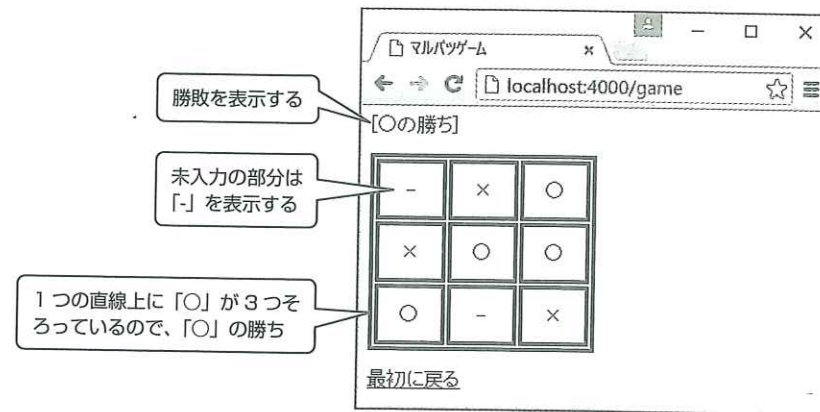


図 13.5 勝敗が付くケース

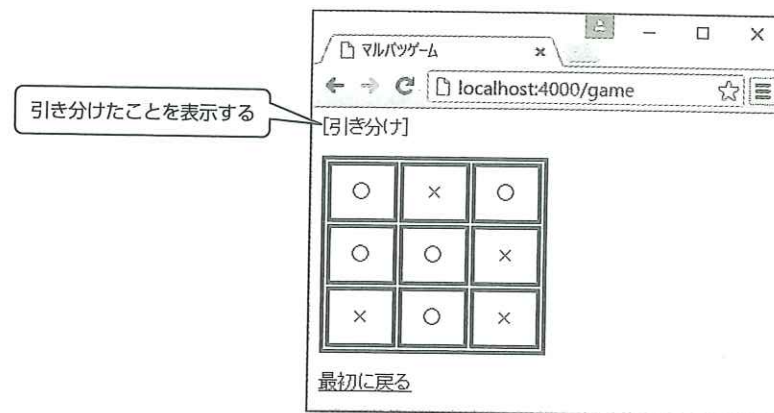


図 13.6 引き分けのケース

192.168.109.1

第 14 章 ◀

並行処理

Go 言語の大きな特徴の 1 つとして、言語自体が並行処理をサポートしていることが挙げられます。Go 言語の並行処理は、「ゴルーチン」と「チャンネル」という 2 つの要素で実現します。本章では、このゴルーチンとチャンネルによる並行処理について説明します。